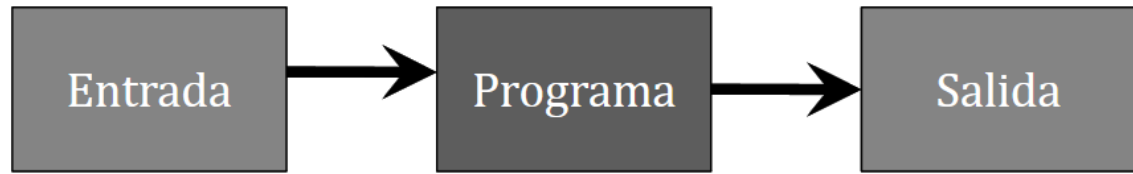


Argumentos del main

PROFESOR: ING. WEBER FEDERICO

Introducción



- La entrada es la “información” o datos que queremos analizar o utilizar.
- Luego el programa realiza una secuencia de pasos y operaciones sobre estos datos de entrada.
- Al finalizar el proceso, obtenemos un resultado.

- En general, hasta ahora usábamos como entrada lo que ingresábamos por teclado. Pero también puede ser un archivo, datos de red, o información que le envía el SO al programa cuando lo ejecutamos.
- **Esto último** es lo que vamos a ver ahora.

Introducción

- Frecuentemente usamos comandos como estos:
 - `gcc -o programa main.c`
 - `git commit -m "primer commit"`
- ¿Cómo llega "main.c" al programa gcc para que sepa qué compilar?
- ¿Cómo recibe git el texto "primer commit" que escribimos?
- **La respuesta está en los argumentos del main.**
- Cada uno de esos textos que escribimos después del nombre del programa llega a la función `main()` como un parámetro.
- Hoy vamos a aprender a recibir y usar esa información.

Función main ()

- En C, **main** puede definirse con o sin parámetros. Cuando se incluyen parámetros, la definición convencional es:

```
int main(int argc, char *argv[])
```

- Estas dos variables nos permiten acceder a los argumentos de la línea de comandos. Por convención se les nombra **argc** y **argv**.
- ¿desde donde se le envían parámetros a **main**?
 - Desde el SO, por ejemplo desde Windows:

```
C:\>miProg.exe arg1 arg2 "arg con espacios" arg4
```

En este caso los parámetros los envía el usuario que ejecuta el programa, podría ser un script.

argc y argv

```
int main(int argc, char *argv[])
```

- **argc** (abreviatura de *argument count*, "recuento de argumentos"): es un entero (int) que indica **cuántos argumentos** se pasaron al programa en la línea de comandos.
 - Este conteo *incluye* el nombre del programa en sí mismo como primer argumento.
 - En otras palabras, argc es normalmente al menos 1, porque **siempre** cuenta la invocación del programa
 - Por ejemplo, si ejecutamos el programa sin argumentos adicionales, argc será 1 (solo el propio nombre).
 - Y en este caso:

```
C:\>miProg.exe arg1 arg2 "arg con espacios" arg4
```

argc = 5 (nombre de programa + 4 arg)

argc y argv

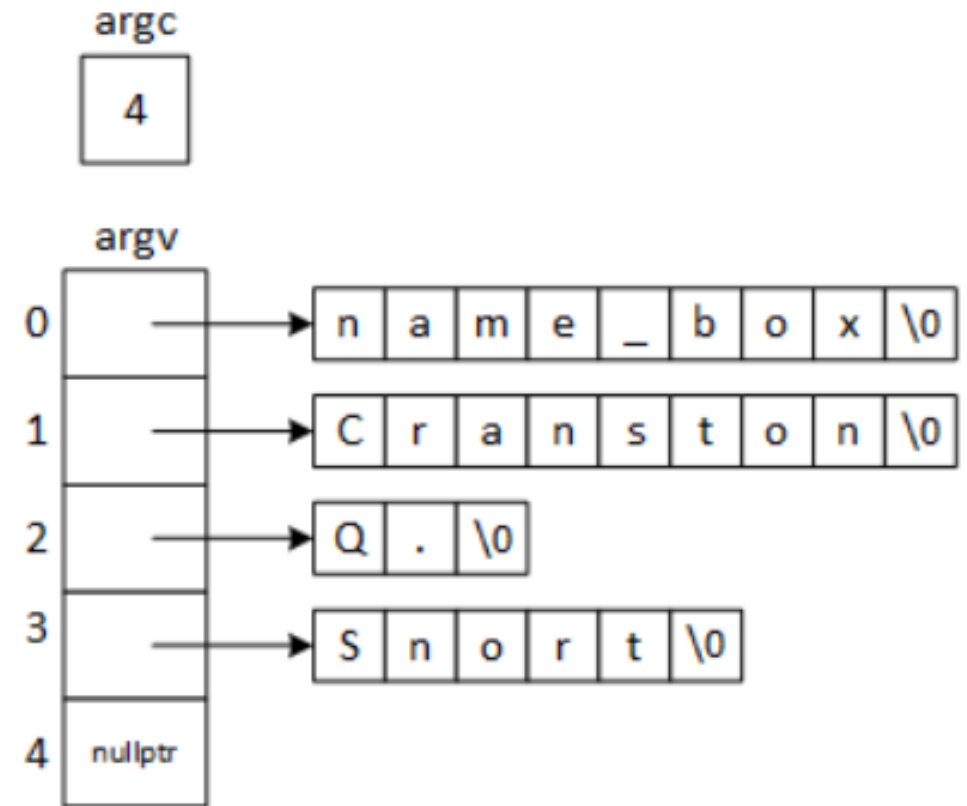
```
int main(int argc, char *argv[])
```

- argv (abreviatura de argument vector, "vector de argumentos"): es un vector de strings que contiene los argumentos proporcionados.
 - En C, esto se representa como `char *argv[]` (vector de punteros a char) o equivalente `char **argv` (puntero a punteros a char).
 - Cada elemento `argv[i]` es un string C (es decir, una secuencia de caracteres terminada en `'\0'`) con uno de los argumentos.
 - El vector tiene `argc` elementos válidos indexados de 0 a `argc-1`. Por convenio:
 - `argv[0]` es el nombre con el que se ejecutó el programa (puede ser el nombre del ejecutable, por ejemplo "miPrograma", o una ruta como `"/miPrograma"` dependiendo del sistema)
 - `argv[1]` es el primer argumento "real" pasado al programa, `argv[2]` el segundo, y así sucesivamente.
 - `argv[argc]` está definido por el estándar C/C++ como un puntero nulo (NULL), que actúa como centinela para marcar el fin del vector de argumentos. Aunque no se suele usar explícitamente en C (ya que conocemos `argc`), esta posición nula existe.

Visualización de cómo se almacenan estos argumentos en memoria

```
C:\>name_box Cranston Q. Snort
```

- Los argumentos en **argv** son proporcionados al programa por el **sistema operativo** en el momento de la ejecución del programa.
- De hecho, cuando el SO invoca nuestro programa, llama a **main** y le pasa estos dos parámetros (**argc** y **argv**) con la información de la línea de comandos.
- Esto permite que nuestros programas reaccionen a información externa al momento de iniciarse (por ejemplo, pasando un nombre de archivo a procesar, opciones, etc.), en lugar de tener valores fijos o pedir todos los datos por entrada estándar después de iniciar.



Equivalencia de `char *argv[]` y `char **argv`:

- Ambas notaciones en la declaración de `main` significan lo mismo.
- En C, declarar un parámetro como `char *argv[]` (vector de punteros a char) es equivalente a `char **argv` (puntero a puntero a char), ya que los vectores como parámetros decaen a punteros.
- Es puramente cuestión de estilo; veremos ambas formas en código de distintos autores.

```
int main(int argc, char **argv)
```

```
int main(int argc, char *argv[])
```

son firmas idénticas en la práctica (el estándar C99 y C11 permiten ambas). Lo importante es que `argv` finalmente se usa como si fuera un vector indexable de strings.

Variaciones validas de la función main:

- El estándar ISO C define dos firmas principales para main en un programa

```
int main()
```

```
int main(void)
```

```
int main(int argc, char **argv)
```

```
int main(int argc, char *argv[])
```

- En todos los casos, **main** debe devolver un **int**, que corresponde al **código de salida** del programa (convención: retornar 0 indica terminación exitosa, cualquier otro valor indica algún error o código particular).
- Si el programa llega al final de **main** sin un **return**, desde el estándar C99+ implica un **return 0** automático.

ejemplo01:

```
#include <stdio.h>

int main(int argc, char *argv[]) {
    printf("Numero de argumentos (argc): %d\n", argc);

    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }
    return 0;
}
```

ejemplo01.c en github

Validación de argumentos y buenas prácticas:

- Siempre que el programa espere un número específico de argumentos, es importante verificar argc.
- Si argc no coincide con lo esperado (por ejemplo, faltan argumentos obligatorios), se debe notificar al usuario e idealmente mostrar uso correcto del programa.
- Por ejemplo, si un programa espera 2 argumentos (además del nombre), argc debería ser 3. Podemos hacer:

```
if (argc < 3) {  
    printf ("Se esperaban 3 argumentos y se recibieron %d\n", argc);  
    return 1;  
}
```

Convención `--help` / `-h`

- Es una convención universal: cuando el usuario pasa `--help` o `-h`, el programa muestra cómo usarlo y termina.
- Ejemplos que ya conocen:
- `gcc --help` `git --help` `ls --help`
- Implementación básica:

```
int main(int argc, char *argv[]) {
    if (argc > 1 && (strcmp(argv[1], "--help") == 0 || strcmp(argv[1], "-h") == 0)) {
        printf("Uso: %s <numero1> <numero2>\n", argv[0]);
        return 0;
    }
    if (argc < 3) {
        printf("Error: se esperan 2 argumentos.\n");
        return 1;
    }
    // ... resto del programa
}
```

`ejemplo02.c` en github

Validación de argumentos usando atoi :

- **atoi – ASCII to Integer**
 - Función simple para convertir una cadena (char *) en un número entero (int).
 - Declarada en la biblioteca <stdlib.h>.
 - No detecta errores: si la cadena no es un número válido, devuelve 0 sin avisar.
 - Es la que más usamos en INFO II, pero no recomendada en programas críticos.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Uso: %s <numero>\n", argv[0]);
        return 1;
    }

    int valor = atoi(argv[1]);

    printf("[atoi] Valor convertido: %d\n", valor);
    return 0;
}
```

[ejemplo03.c en github](#)

Validación de argumentos – usando strtol :

strtol – String to Long

- Convierte una cadena a número, pero con más control que atoi.
- Declarada en <stdlib.h>.
- Permite detectar errores de conversión (con errno y endptr).
- Soporta distintas bases: decimal, hexadecimal, octal, etc.

```
long int strtol(const char *nptr, char **endptr, int base);
```

Parámetros:

- ***nptr**: puntero a la cadena de caracteres que querés convertir.
- ****endptr**: puntero a puntero donde strtol va a guardar la posición del primer carácter no válido (se puede pasar NULL si no te interesa).
- **base**: la base numérica para la conversión (ej. 10 para decimal, 16 para hexadecimal).

Validación de argumentos usando strtol :

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Falta un número como argumento.\n");
        return 1;
    }
```

```
    char *fin;
```

```
    long valor = strtol(argv[1], &fin, 10);
```

```
    if (*fin != '\0') {
```

```
        printf("El texto '%s' no es un numero valido.\n", argv[1]);
```

```
    } else {
```

```
        printf("Numero convertido: %ld\n", valor);
```

```
    }
```

```
    return 0;
}
```

Fin es un puntero a carácter.

La función strtol esta esperando un **char.

Le envio la dirección de mi puntero fin. Eso le permite a strtol modificar el valor del puntero, y hacer que apunte al carácter posterior al número interpretado.

[ejemplo04.c en github](#)

El contenido de lo apuntado por fin

Validación de argumentos usando atof :

atof – ASCII to Float

- Función simple para convertir una cadena (char *) en un número decimal (double).
- Declarada en la biblioteca <stdlib.h>.
- No detecta errores: si la cadena no es un número válido, devuelve 0 sin avisar.
- Es la que más usamos en INFO II, pero no recomendada en programas críticos.

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Error: falta un numero.\n");
        return 1;
    }

    float valor = atof(argv[1]); // Convierte sin verificar
    printf("[atof] Valor convertido: %.2f\n", valor);

    return 0;
}
```

[ejemplo05.c en github](#)

Validación de argumentos – usando strtod :

strtod – String to Double

- Convierte una cadena a número, pero con más control que atof.
- Declarada en <stdlib.h>.
- Permite detectar errores de conversión (con errno y endptr).

Parámetros:

- ***nptr**: puntero a la cadena de caracteres que querés convertir.
- ****endptr**: puntero a puntero donde strtod va a guardar la posición del primer carácter no válido (se puede pasar NULL si no te interesa).

```
double strtod(const char *nptr , char **endptr);
```

Validación de argumentos usando strtod:

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Error: falta un número.\n");
        return 1;
    }

    char *fin;
    float valor = strtod(argv[1], &fin);

    if (*fin != '\0') {
        printf("Error: '%s' no es un numero valido.\n", argv[1]);
        return 1;
    }

    printf("[strtod] Valor convertido: %.2f\n", valor);
    return 0;
}
```

ejemplo06.c en github

Convenciones de flags y getopt

- En el mundo real, los argumentos siguen convenciones:

- `-o archivo` flag corto (una letra, un guión)
- `--output archivo` flag largo (palabra, dos guiones)
- `-v` flag booleano (sin valor)
- `-Wall -Wextra` múltiples flags combinados

- Ejemplos que ya conocen:

- `gcc -Wall -o salida main.c`
- `git commit -m "mensaje" --amend`
- `ping -c 4 google.com`

- Esto se puede parsear a mano con `strcmp` en un `for` sobre `argv`.
- Pero cuando hay muchos flags, la biblioteca estándar POSIX ofrece:

- `getopt()` para flags cortos (`-v`, `-o archivo`)
- `getopt_long()` para flags largos (`--verbose`, `--output archivo`)

- No las vamos a usar en INFO II, pero sepan que existen.
- Cuando escriban programas con muchas opciones, no reinventen la rueda.

Tabla comparativa

Función	Prototipo completo	Valida errores	Uso principal
atoi	int atoi (const char *nptr);	No	Convertir texto a entero
strtol	long int strtol (const char *nptr , char** endptr , int base);	Sí	Conversión robusta a entero
atof	double atof (const char *nptr);	No	Convertir texto a double
strtod	float strtod (const char *nptr , char** endptr);	Sí	Conversión robusta a float

- Si bien no lo vamos a usar en este curso es importante destacar que **atoi** y **atof** no modifican la variable **errno**.
- **errno** es una variable global definida en **<errno.h>**, que indica si ocurrió un error en algunas funciones de la biblioteca estándar, como **strtol**, **strtod**, **fopen**, **malloc**, entre otras.

Variables de entorno y su uso en C

- Ya vimos cómo pasarle datos al programa en el momento de ejecutarlo. Pero ¿qué pasa con configuración que ya existe en el sistema antes de que el programa arranque?
- Además de argc/argv, los programas en C pueden obtener información de **variables de entorno** del sistema operativo.
- Las variables de entorno son pares clave=valor que el entorno (generalmente la shell o el SO) mantiene y que se heredan por los procesos hijos.
- A diferencia de los argumentos por línea de comando, las variables de entorno no pasan por argv (a menos que se haga explícitamente mediante extensiones); se accede a ellas a través de funciones específicas.

Variables de entorno y su uso en C

¿Qué son las variables de entorno?

- Son valores nombrados que existen en el SO antes de lanzar el programa, y que el SO pone a disposición del programa.
- Por ejemplo, en sistemas UNIX es típico tener variables como HOME (directorio de inicio del usuario), PATH (rutas de búsqueda de ejecutables), LANG (configuración de idioma), etc.
- En Windows existen SystemRoot, USERNAME, TEMP, etc.
- Todas son cadenas de caracteres (texto).
- El programa padre (por ejemplo, la shell) provee estas variables al iniciar el programa hijo. Por lo tanto, **desde la perspectiva de C, son datos globales que podemos consultar en cualquier momento durante la ejecución.**

Acceso a variables de entorno en C

- La forma estándar de obtener el valor de una variable de entorno es usando la función **getenv** (definida en `<stdlib.h>`).
- Esta función recibe el nombre de la variable (una cadena) y devuelve un puntero a la cadena de valor correspondiente, o NULL si no existe tal variable. Ejemplo:

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {
    char *user = getenv("USERNAME");
    if (user != NULL) {
        printf("La variable USERNAME es: %s\n", user);
    } else {
        printf("USERNAME no está definida.\n");
    }
    return 0;
}
```

```
>argmain.exe
La variable USERNAME es: Fede
```

Acceso a variables de entorno en C

- Muchos programas usan `getenv` para leer configuraciones como HOME (por ejemplo, para saber en qué directorio crear un archivo de config del usuario) u opciones como TZ (zona horaria), etc.
- Es útil pensar en las variables de entorno como un segundo canal de entrada además de `argv`. De hecho, el POSIX estándar (acuerdo de paz en sistemas UNIX, como Linux y MAC) sugiere que hay dos mecanismos de pasar información al nuevo proceso: los argumentos (`argv`) y el entorno. Ambos se configuran en el momento de la llamada del SO para lanzar un nuevo programa.

```
#include <stdlib.h>
#include <stdio.h>

int main(int argc, char *argv[]) {

    char *verbose = getenv("VERBOSE");
    if (verbose != NULL) {
        printf("La variable VERBOSE es: %s\n", verbose);
    } else {
        printf("VERBOSE no esta definida.\n");
    }
    return 0;
}
```

ejemplo07.c en github
ejemplo08.c en github

Uso del tercer parámetro envp

- Algunas implementaciones permiten declarar main con tres parámetros:
 - `int main(int argc, char *argv[], char *envp[]).`
- Aquí **envp** es un vector de strings similar a argv, que contiene todas las variables de entorno en formato "NOMBRE=valor". Termina con un elemento NULL también.
- Esto no es parte del estándar ISO C, pero es una extensión común en Unix (GCC lo soporta).
- Por ejemplo, podríamos iterar sobre envp para imprimir todas las variables. Sin embargo, dado que getenv existe, no es necesario depender de envp.
- De hecho, getenv es más portátil (funciona en Windows y Unix), mientras que envp en main puede que no esté disponible en ciertos compiladores.

Ejercicios de argumentos del main y variables del sistema

1. Realizar un programa que muestre la cantidad de argumentos del main que recibe y los enumere.
2. Realizar un programa que reciba un único argumento desde la línea de comandos. Si el argumento es un número entero válido y positivo, imprimirlo por stdout. Si el argumento no es válido o es negativo, imprimir un mensaje de error por stderr. Redirigir el resultado a desde el Shell a un archivo "resultado.txt"
3. Realizar un programa que muestre todas las variables de "environment" del sistema.
4. Realizar un programa que funcione como una calculadora por argumentos del main. Así:
calc operacion numero1 numero2 ...
5. **(versión UNIX)**
En Bash, el shell que funciona en las terminales de linux las variables de environment se generan así:
`export VAR="contenido de la variable"`
(versión Windows)
En CMD, el shell que funciona en las terminales de windows las variables de environment se generan así:
`set VAR="contenido de la variable"`
(para ambos) Realizar un programa que muestre el contenido de una variable MIVAR.
6. Realizar un programa que devuelva la suma de sus argumentos a traves del return.
Probarlo de esta forma en consola:
(versión UNIX)
`./suma 3 6`
`echo La suma de 3 y 6 es $? [El resultado del ultimo programa queda guardado en la variable $? ]`
(versión Windows)
`suma.exe 3 6`
`echo %ERRORLEVEL% [El resultado del ultimo programa queda guardado en la variable %ERRORLEVEL% ]`

FIN

